Métricas DORA em projetos open-source: o que se pode medir com dados públicos do GitHub

João Santana Lopes[1], **Pedro Maia**[1], **Arthur Candian**[1]

[1]Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Laboratório de Experimentação de Software

joaovslopes20@gmail.com, pedromaiasilva17@gmail.com,
acandian15@gmail.com

Abstract. *The DORA metrics describe software delivery performance through four indicators, but they were established from self-reported surveys of corporate teams, which makes them neither auditable nor obviously applicable to open-source projects. We compute the four metrics for 350 popular GitHub repositories over a fixed 12-month window, using only public API data, with releases as a proxy for deployments and failed CI runs as a proxy for failed changes. We validate the proxies against a gold sample of 60 repositories and 299 releases labelled independently by three raters. The median deployment frequency is 0.42 releases per week and the median recovery time is 2.56 hours, but the change failure rate depends critically on how failure is operationalised: the CI proxy and the delivery proxy correlate at only* $\rho = 0.107$ *and disagree on the DORA category of 20.6% of the sample. A sensitivity analysis over three definitions of deployment moves between 25% and 33% of repositories across categories. We conclude that DORA classifications derived from public repository data are contingent on operational choices that are rarely made explicit.*

Resumo. *As métricas DORA descrevem o desempenho de entrega de software por quatro indicadores, mas foram estabelecidas a partir de questionários autorrelatados por equipes corporativas, o que as torna não auditáveis e de aplicação incerta a projetos open-source. Calculamos as quatro métricas para 350 repositórios populares do GitHub em uma janela fixa de 12 meses, usando apenas dados públicos da API, com releases como proxy de deploy e falhas de CI como proxy de falha. Validamos os proxies contra uma amostra-ouro de 60 repositórios e 299 releases, rotuladas de forma independente por três avaliadores. A mediana da frequência de deploy é de 0,42 release por semana e a do tempo de recuperação é de 2,56 horas, mas a taxa de falha depende de como a falha é operacionalizada: o proxy de CI e o de entrega correlacionam-se a apenas* $\rho = 0{,}107$ *e discordam sobre a categoria DORA de 20,6% da amostra. Uma análise de sensibilidade sobre três definições de deploy move entre 25% e 33% dos repositórios de categoria. Concluímos que classificações DORA obtidas de dados públicos são contingentes a escolhas operacionais raramente explicitadas.*

1. Introdução

As métricas DORA, propostas pelo programa *DevOps Research and Assessment* [DORA 2024], descrevem o desempenho de entrega de uma equipe de software

por quatro indicadores: a frequência de *deploy*, o *lead time* para mudanças, a taxa de falha das mudanças e o tempo de recuperação após uma falha. Com os quatro, a equipe é classificada como *Elite*, *High*, *Medium* ou *Low*, e a literatura associa as categorias superiores a práticas como integração contínua, automação de testes e implantação em pequenos lotes [Forsgren et al. 2018].

Essas métricas foram estabelecidas a partir de questionários respondidos por profissionais, e isso limita seu uso fora desse contexto. Os dados originais são autorrelatados e privados, então ninguém consegue auditá-los nem reproduzi-los. Além disso, as faixas de desempenho foram calibradas em equipes corporativas que praticam entrega contínua, e não se sabe o que elas significam quando aplicadas a projetos *open-source*, cujo ritmo de publicação é decidido por mantenedores voluntários.

Neste trabalho calculamos as quatro métricas apenas com dados públicos do GitHub, sobre uma amostra de 350 repositórios populares que usam GitHub Actions, em uma janela fixa de 12 meses. Como a API não informa quando uma mudança chegou à produção nem quando um defeito afetou usuários, usamos dois *proxies*: a publicação de uma *release* aproxima o *deploy*, e a falha de um *workflow* de integração contínua aproxima a falha de uma mudança. Toda a coleta é reproduzida por um único comando, e o conjunto de dados resultante é publicado junto com o código.

Como os valores dependem desses *proxies*, também os verificamos. Sorteamos uma amostra-ouro de 60 repositórios e 299 *releases*, que os três autores julgaram de forma independente; medimos a concordância entre eles e avaliamos contra esse consenso a heurística automática que identifica *releases* corretivas. Por fim, comparamos três definições alternativas de *deploy* para ver o quanto as conclusões mudam com elas, e a discussão usa bastante esses resultados.

1.1. Questões de pesquisa

RQ 01 Com que frequência são feitos *deploys* nesses projetos?
RQ 02 Qual o *lead time* para mudanças, nas variantes (a) e (b)?
RQ 03 Qual a taxa de falha das mudanças, nas variantes (a) e (b)?
RQ 04 Qual o tempo de recuperação após uma falha?
RQ 05 Frequência de *deploy* e taxa de falha se correlacionam?
RQ 06 Linguagem, popularidade, número de contribuidores, idade e tipo do projeto explicam diferenças no desempenho?
RQ 07 A classificação muda se a unidade de *deploy* for definida de outra forma?
RQ 08 Qual a proporção de *releases* corretivas (*rework rate*) e como ela se compara ao CFR (b)?

Nas RQ 01 a RQ 04 reportamos a mediana e o intervalo interquartil de cada métrica, além da distribuição das categorias DORA.

1.2. Estrutura do artigo

A Seção 2 descreve a metodologia: a fonte de dados e a seleção da amostra, as definições operacionais de cada métrica e a heurística do CFR (b), e o protocolo da validação manual. A Seção 3 apresenta os resultados por questão de pesquisa. A Seção 4 discute o que as hipóteses previam e o que foi observado. A Seção 5 reúne as ameaças à validade nas quatro categorias clássicas [Wohlin et al. 2012], e a Seção 6 apresenta as conclusões.

1.3. Hipóteses

RQ 03: taxa de falha das mudanças. Esperamos que a variante de CI (a) fique, na mediana, entre 10% e 25%. Como os *runs* considerados são disparados por *push* no *default branch*, boa parte das mudanças já passou pela validação de um *pull request* antes do *merge*. Ainda assim, problemas de integração, testes instáveis (*flaky*) e dependências externas devem gerar uma parte das falhas. Para a variante de entrega (b), esperamos valores parecidos ou menores e uma distribuição com muitos repositórios em 0%. Também esperamos que as duas variantes concordem pouco entre si, porque a variante (a) mede falha de *pipeline* e a (b) aproxima falha percebida pelo usuário.

RQ 04: tempo de recuperação. Esperamos uma mediana da ordem de algumas horas, o que colocaria a maioria dos repositórios na faixa *High*. Em projetos ativos, a falha costuma ser resolvida pelo próximo *push*. Esperamos, porém, uma cauda longa e um IQR largo, de minutos a vários dias, porque mantenedores de projetos *open-source* trabalham de forma voluntária e assíncrona e alguns *workflows* rodam raramente. Pelo mesmo motivo, esperamos uma proporção não trivial de episódios censurados.

RQ 05: frequência de *deploy* $\times$ taxa de falha. Para a variante (a), esperamos uma correlação de Spearman fraca e negativa ou nula, em linha com a afirmação do DORA de que velocidade e estabilidade não são um *trade-off*. Para a variante (b), esperamos uma correlação positiva, em parte como artefato da métrica: quem publica muitas *releases* tem mais chances de publicar um *patch* em até 7 dias após qualquer *release*. Se isso se confirmar, a resposta da RQ 05 depende da definição operacional, assim como a análise de sensibilidade da RQ 07 investiga.

2. Metodologia

2.1. Fonte de dados e coleta

Todos os dados vêm da API REST do GitHub, consultada por um *pipeline* próprio em Python que usa apenas a biblioteca `requests`. A coleta inteira é reproduzida por um único comando. Cada resposta da API é gravada em um *cache* local indexado pelo *hash* da URL, de modo que uma coleta interrompida continua de onde parou. A execução final fez 11.989 chamadas à API e reaproveitou 58.459 respostas do *cache*.

A busca de repositórios devolve no máximo 1.000 itens por consulta, e só a faixa de 1.000 a 1.500 estrelas tem 20.493 repositórios. Por isso, dividimos a busca em sete faixas de estrelas (1.000–1.500, 1.500–2.500, 2.500–4.000, 4.000–7.000, 7.000–12.000, 12.000–25.000 e mais de 25.000). De cada faixa entraram os 1.000 repositórios mais estrelados, num total de 7.000 candidatos. Os candidatos foram avaliados alternando as faixas, para que a amostra não saísse inteira da faixa mais popular.

O cliente HTTP respeita a cota da API, esperando o *reset* quando ela acaba, e repete erros 5xx e falhas de rede com *backoff* exponencial (1, 2, 4, 8 e 16 s), até seis tentativas. Todas as listagens são paginadas pelo cabeçalho `Link`. A listagem de *workflow runs* também devolve no máximo 1.000 resultados, e o seu `total_count` satura em 2.500. Por isso, os *runs* são pedidos mês a mês, e o mês que passa do teto é dividido ao meio até caber nele.

2.2. Janela de observação

A janela tem 12 meses, de 01/10/2025 a 30/09/2026, inclusiva nas duas pontas, e é a mesma para todos os repositórios. Só entram releases publicadas e *workflow runs* criados dentro dela, e só no *default branch*. A idade do repositório é medida até o fim da janela, e não até a data da execução, para que uma nova execução chegue ao mesmo valor.

2.3. Seleção da amostra

Os candidatos passaram por filtros em sequência. Cada repositório parou no primeiro filtro em que falhou, e por isso tem um único motivo de descarte. A coleta parou ao completar 350 repositórios aprovados, acima do mínimo de 300. A Tabela 1 mostra o funil.

Tabela 1. Funil de seleção da amostra.

Etapa	Restantes	Descart.	Motivo do descarte
Candidatos da busca (7 faixas)	7.000	–	–
Avaliados	1.565	5.435	não avaliados (amostra completa)
Usam GitHub Actions	1.057	508	nenhum workflow
≥ 5 releases na janela	439	618	menos de 5 releases
Runs mensuráveis	433	6	> 1.000 *runs* em um dia
≥ 50 *runs* válidos	352	81	menos de 50 *runs* válidos
Coleta sem erro	350	2	erro definitivo da API
Amostra final	**350**	–	–

Os 5.435 candidatos não avaliados nunca foram consultados, porque a coleta parou ao fechar a amostra; eles não foram reprovados por critério algum. Os seis repositórios com mais de 1.000 *runs* em um único dia foram descartados porque não há como recuperar todos os *runs* desses dias, nem saber quantos faltam. Mantê-los faria o CFR (a) e o tempo de recuperação serem calculados sobre dados truncados. Com isso, a amostra exclui os projetos de maior volume de CI. Os dois erros definitivos foram um HTTP 504 recorrente e uma listagem de releases que passou do limite de 10.000 resultados paginados da API.

A amostra final reúne 350 repositórios com 1.493 a 291.317 estrelas (mediana de 11.944), concentrados nas faixas mais populares: 97 têm mais de 25.000 estrelas e 27 estão na faixa de 1.000 a 1.500. As linguagens mais frequentes são TypeScript (73), Python (72), Go (45), Rust (39) e JavaScript (28). A mediana é de 22 releases e 688 *runs* válidos por repositório na janela, e a idade mediana é de 6,0 anos.

A amostra não é aleatória. Dentro de cada faixa, só os 1.000 repositórios mais estrelados chegaram a ser candidatos. Além disso, as faixas foram avaliadas na mesma proporção, mas os repositórios mais populares passaram mais nos filtros, e por isso a amostra final pende para eles. Os resultados valem para repositórios populares que usam GitHub Actions e publicam releases, e não para o GitHub como um todo.

2.4. Dicionário de dados

O *pipeline* gera dois arquivos. O `repositorios.csv` tem uma linha por repositório avaliado, aprovado ou descartado, e o `funil.csv` tem uma linha por etapa da seleção.

O dicionário completo, com tipo, unidade e origem de cada coluna, está no repositório do trabalho. As colunas de `repositorios.csv` formam cinco grupos: (i) metadados (estrelas, linguagem, *default branch*, idade e contribuidores); (ii) contagens da seleção (releases na janela, sem rascunhos e pré-releases, e *runs* válidos, isto é, de `push` no *default branch* que terminam em sucesso ou falha); (iii) as métricas DORA em todas as variantes, com as contagens da heurística de release corretiva e a proporção de episódios censurados; (iv) a nota de 1 a 4 por métrica e a categoria geral; e (v) o motivo do descarte. Valor vazio significa *não medido*, e não zero; métrica vazia não recebe nota e fica fora da mediana da categoria geral.

2.5. Definições operacionais

Como o GitHub não registra *deploys* nem falhas em produção, todas as métricas são medidas por *proxies*. Consideramos *deploy* uma release publicada (`draft = false`) no *default branch*; pré-releases e tags entram apenas como variantes na análise de sensibilidade. A frequência de deploy é o número de releases na janela dividido pelo número de semanas da janela.

O *lead time* é calculado em duas variantes a partir de `compare/{anterior}...{R}` e de `commit.author.date`: (a) por release, pela diferença entre a data da release e a do commit mais antigo incluído; (b) por commit, sobre todos os commits de todas as releases. Releases sem release anterior ou com `compare` em 404 são ignoradas e contabilizadas.

A *change failure rate* (CFR) também tem duas variantes: (a) a proporção de execuções de CI com falha, que mede falha de *pipeline* e não de produção; (b) a proporção de releases seguidas de uma release corretiva em até 7 dias.

2.6. CFR (b): proxy de entrega

Uma release R é considerada falha se for seguida, em até 7 dias, por uma release corretiva. O CFR (b) é o número de releases que falharam dividido pelo número de releases avaliadas. Releases publicadas nos últimos 7 dias da janela não podem ser avaliadas e ficam fora do denominador (censura); a quantidade delas é reportada.

Como o GitHub não indica se uma release é corretiva, usamos uma heurística sobre a versão (SemVer) e o assunto dos commits entre as duas releases. R é comparada apenas com a release seguinte; lemos só o assunto do commit; quando o `compare` falha, vale apenas o critério de versão; a primeira release do repositório nunca é corretiva. Testamos quatro versões. As três primeiras foram v1 (só *patch* ou algum commit `fix`/`revert`/`hotfix`), v2 (só *patch* ou ao menos metade dos commits são correções) e v3 (*patch* com algum commit de correção, ou ao menos metade dos commits são correções). A v4, descrita adiante, foi montada olhando os erros dessas três.

2.7. Validação da heurística

Sorteamos 60 repositórios da amostra final (`random_state = 42`) e 5 releases de cada um. Os três integrantes rotularam as releases de forma independente, sem ver a saída da heurística, e o consenso seguiu o protocolo da seção de validação manual. Comparamos a heurística com o consenso por precisão, recall e F1 da classe “corretiva”, aceitando F1 $\geq$ 0,70.

Tabela 2. Desempenho de cada versão da heurística contra o consenso da amostra-ouro (299 *releases*, 111 corretivas).

Versão	VP	FP	FN	VN	Precisão	Recall	F1
v1	110	170	1	18	0,39	0,99	0,56
v2	84	74	27	114	0,53	0,76	0,62
v3	71	67	40	121	0,51	0,64	0,57
v4	81	57	30	131	0,59	0,73	**0,65**

A v4 foi a escolhida, com F1 de 0,65. Nenhuma versão atingiu o limiar de 0,70, e isso é reportado como limitação nas ameaças à validade.

As três primeiras versões partem da mesma suposição: toda mudança de *patch* no SemVer seria correção de defeito. Publicar *patch* é a prática recomendada pelo próprio SemVer e não implica corrigir nada, de modo que a v1 marcou 170 *releases* que o consenso considerou não corretivas. Ela acerta quase todas as corretivas (recall 0,99) ao custo de dizer "sim" para quase tudo (precisão 0,39). A v3 tentou apertar o critério exigindo também um *commit* de correção, mas perdeu corretivas verdadeiras sem ganhar precisão.

A v4 troca o critério isolado de *patch* por dois sinais. O primeiro são as *release notes*: quando elas trazem uma seção de correção e nenhuma de novidade, 88% das *releases* da amostra-ouro são corretivas pelo consenso, contra 56% quando o único sinal é a mudança de *patch*. O segundo é a exigência de pelo menos um *commit* de correção entre os *substantivos*, isto é, descontando manutenção, dependências e infraestrutura (`chore`, `ci`, `docs`, `test`, `bump`). Sem esse desconto, uma *release* de *patch* composta apenas de atualizações automáticas de dependência seria classificada como corretiva, que é o caso mais frequente entre os falsos positivos da v1.

Quase todo o ganho vem da precisão, que sobe de 0,39 na v1 para 0,59: os falsos positivos caem de 170 para 57, com *recall* de 0,73. Permanecem 30 falsos negativos, em sua maioria *releases* grandes de projetos que não seguem SemVer e cujas notas misturam correções e novidades, os mesmos casos em que os avaliadores também divergiram.

Nenhum dos critérios usa limiar numérico ajustado. A regra é composta de condições binárias e pode ser aplicada a outra amostra sem calibração.

2.8. Validação manual

Como as métricas dependem de *proxies* e de uma heurística automática para identificar *releases* corretivas, comparamos parte dos dados com o julgamento humano. Sorteamos uma *amostra-ouro* de 60 repositórios da amostra final (`df.sample(n=60, random_state=42)`, após ordenar os repositórios pelo nome) e, para cada um, 5 *releases* da janela, com a mesma semente. Só entraram no sorteio as *releases* com *release* anterior e comparação de *commits* disponível, porque sem a lista de *commits* não é possível julgar se uma *release* é corretiva. Um repositório tinha menos de 5 *releases* nessa condição, e a amostra ficou com 299 *releases*.

Os três autores rotularam a amostra de forma independente, sem consultar uns aos outros e sem acesso à saída da heurística. Cada um classificou: (i) o tipo do projeto (biblioteca/*framework*, aplicação/serviço, ferramenta CLI ou outro); (ii) se as *releases* do repositório representam entregas reais ao usuário (sim, não ou incerto); e (iii) para cada

release sorteada, se ela é corretiva (sim ou não), a partir das *release notes* e da lista de *commits*. O critério de cada rótulo foi definido por escrito antes da rotulagem.

A concordância entre os avaliadores foi medida pelo kappa de Fleiss em cada dimensão, que desconta a concordância esperada por acaso, e interpretada pela escala de Landis e Koch [Landis and Koch 1977]. A Tabela 3 resume os resultados.

Tabela 3. Concordância entre os três avaliadores na amostra-ouro.

Dimensão	Itens	κ	Concordância
Tipo do projeto	60	0,835	Quase perfeita
Releases são entregas reais	60	0,745	Substancial
Release corretiva	299	0,876	Quase perfeita

O menor kappa é o de "releases são entregas reais" (0,745), e o motivo é o desbalanceamento dos rótulos, não a ambiguidade do critério: o consenso classificou 59 dos 60 repositórios como "sim" e apenas um como "incerto", de modo que a concordância esperada por acaso é muito alta e uma única divergência já derruba o coeficiente, embora a concordância bruta seja de 98,3%. O kappa se comporta assim sempre que os rótulos se concentram numa categoria, e por isso reportamos também a distribuição dos rótulos.

A dimensão mais ambígua é o tipo do projeto (0,835), com 9 divergências em 60, concentradas em dois limites: o que separa biblioteca de aplicação em projetos que entregam ao mesmo tempo um servidor e um SDK, e o alcance da categoria "outro". Na dimensão que interessa à heurística, release corretiva, o kappa é de 0,876, com 26 divergências em 299: o critério não é inteiramente objetivo nem para humanos, porque releases que misturam correções e novidades admitem leitura dupla. Como a heurística automática é avaliada contra o consenso desses mesmos avaliadores, esse valor serve de teto prático para ela.

O *rótulo de consenso* foi definido por maioria simples (pelo menos 2 dos 3 avaliadores). Os casos sem maioria (empates 1-1-1, possíveis nas dimensões com mais de duas categorias) seriam decididos em reunião dos três autores, mas não houve nenhum. O consenso sobre o tipo do projeto é usado como fator na RQ 06, e o consenso sobre *releases* corretivas é a referência para avaliar a heurística do CFR (b).

3. Resultados

3.1. Visão geral

A amostra final tem 350 repositórios. Como as quatro métricas têm distribuições assimétricas e cauda longa, reportamos mediana e intervalo interquartil (IQR), e não média e desvio padrão. A Tabela 4 resume as seis variáveis.

3.2. RQ 01: frequência de deploy

A mediana é de 0,42 *release* por semana, cerca de uma a cada duas semanas e meia, com IQR de 0,84. A distribuição é fortemente assimétrica à direita (Figura 1): 260 repositórios (74,3%) publicam menos de uma *release* por semana e apenas 5 (1,4%) alcançam o patamar *Elite* de sete ou mais por semana. O máximo observado é de 49,6.

Tabela 4. Estatística descritiva das métricas DORA (350 repositórios).

Métrica	n	Q1	Mediana	Q3	IQR
Frequência de deploy (releases/semana)	350	0,19	0,42	1,04	0,84
Lead time (a) (horas)	350	50,8	251,7	848,4	797,6
Lead time (b) (horas)	350	65,4	284,0	845,4	780,1
CFR (a), proxy de CI	350	0,021	0,049	0,121	0,100
CFR (b), proxy de entrega	350	0,052	0,200	0,416	0,364
Tempo de recuperação (horas)	330	0,98	2,56	9,53	8,55

Nesta dimensão a amostra vai pior que nas outras: 107 repositórios (30,6%) ficam em *Low* e 153 (43,7%) em *Medium*. Os cortes DORA foram concebidos para *deploys* em ambientes de entrega contínua, enquanto uma *release* publicada no GitHub é um marco de versionamento, muitas vezes mensal ou trimestral por decisão do projeto. A frequência baixa reflete mais a prática de versionamento do ecossistema open-source do que uma capacidade de entrega limitada.

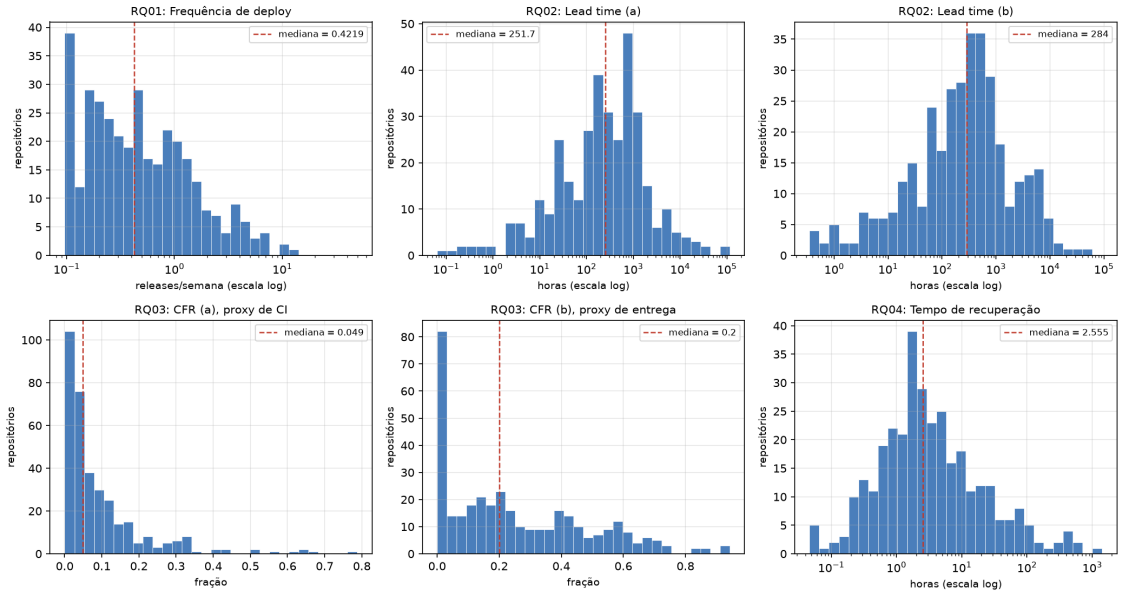


Figura 1. Distribuição das seis métricas nos 350 repositórios. Frequência, *lead time* e tempo de recuperação em escala logarítmica; a linha tracejada marca a mediana.

3.3. RQ 02: lead time para mudanças

As duas variantes dão resultados próximos: 251,7 h (10,5 dias) na variante (a), que mede da publicação da *release* até o *commit* mais antigo nela, e 284,0 h (11,8 dias) na variante (b), que considera todos os *commits*. Os IQRs são semelhantes (797,6 h e 780,1 h), as caixas se sobrepõem quase inteiramente (Figura 2) e a correlação de Spearman entre elas é de 0,744.

Para a RQ 02, portanto, escolher uma ou outra variante muda pouco a conclusão. A dispersão é grande nas duas (de 0,06 h a $1{,}1 \times 10^5$ h na variante (a)): há projetos que publicam a partir de um ramo recente e outros que acumulam *commits* por meses antes de

publicar. Na classificação, a distribuição é relativamente equilibrada: 57 em *Elite*, 89 em *High*, 103 em *Medium* e 101 em *Low*.

3.4. RQ 03: taxa de falha das mudanças

Aqui as duas variantes divergem mais do que em qualquer outra métrica, e as hipóteses da Introdução erraram nas duas direções.

A variante (a), proxy de CI, tem mediana de 0,049 (4,9%) e IQR de 0,100. A hipótese previa uma mediana entre 10% e 25%: o observado é menos da metade do piso previsto, e 13 repositórios (3,7%) não têm nenhum *run* com falha na janela. A explicação provável é que os *runs* contados são disparados por *push* no *default branch*, isto é, mudanças já validadas por um *pull request* antes do *merge*: mede-se a taxa de falha do que já foi aprovado, e não a de todas as mudanças propostas.

A variante (b), proxy de entrega, tem mediana de 0,200 (20,0%) e IQR de 0,364. A hipótese previa valores parecidos ou menores que os da variante (a) e concentração em 0%; o observado é quatro vezes maior que o da variante (a), embora 80 repositórios (22,9%) de fato fiquem em zero.

Confirmou-se a terceira hipótese: as duas variantes concordam pouco. Além de terem medianas diferentes, elas ordenam os repositórios de maneiras diferentes: a correlação de Spearman entre elas é de 0,107 ($p = 0{,}045$), praticamente nula, e um repositório bem avaliado por uma pode ser mal avaliado pela outra. Voltamos a esse problema de constructo na seção de Ameaças à Validade. Pela variante (a), 283 repositórios (80,9%) ficam na faixa *Elite* (CFR $\leq$ 0,15), a nota mais generosa das quatro.

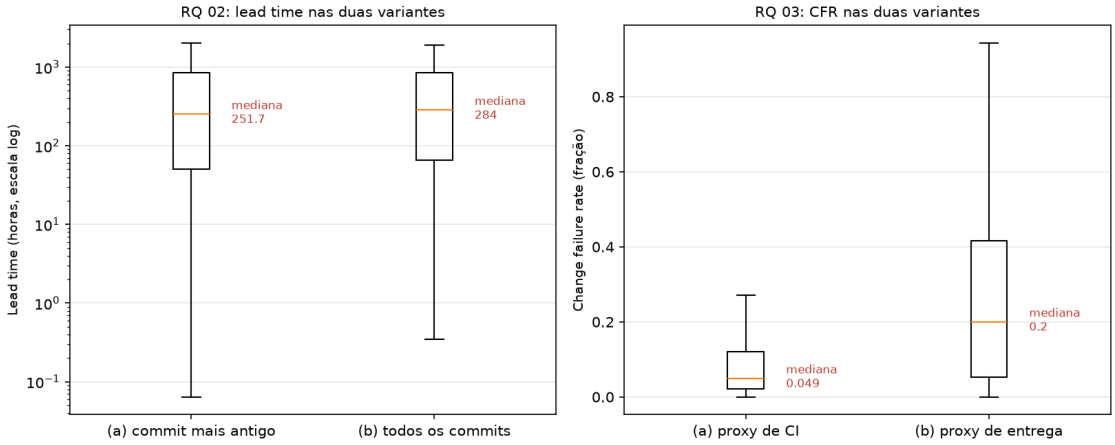


Figura 2. As duas variantes lado a lado. À esquerda, o *lead time*, cujas variantes quase se sobrepõem; à direita, o CFR, em que a mediana da variante (b) é quatro vezes a da (a).

3.5. RQ 04: tempo de recuperação

A mediana é de 2,56 h, com IQR de 8,55 h, calculada sobre os 330 repositórios com ao menos um episódio de falha seguida de sucesso na janela. A hipótese se confirma: previa-se "algumas horas" e a maioria na faixa *High* (de 1 h a menos de um dia), e de fato 196 repositórios (59,4%) caem nessa faixa, contra 86 (26,1%) em *Elite* e 48 (14,5%) em 24 h ou mais. A cauda longa também se confirma: o máximo é de 1.418 h, quase dois meses.

Vinte repositórios (5,7%) não tiveram nenhum episódio na janela e ficam fora do cálculo da mediana, como determina o protocolo. São, em sua maioria, repositórios cujos *workflows* não registraram falha alguma no período, e não repositórios inativos: `arendst/Tasmota`, por exemplo, tem 569 *runs* válidos e nenhuma falha.

Esperávamos uma proporção não trivial de episódios censurados, isto é, que a janela não viu encerrar, e isso não se confirmou. A censura foi desprezível: mediana 0, terceiro quartil 0,0033, e 234 dos 330 repositórios (70,9%) sem nenhum episódio censurado. Quase toda falha iniciada dentro da janela também foi resolvida dentro dela, então a censura pouco afeta a mediana.

3.6. Classificação DORA geral

A categoria de cada repositório é a mediana das quatro notas, arredondada para baixo (Tabela 5 e Figura 3).

Tabela 5. Distribuição da classificação DORA geral.

Categoria	n	%
Elite	25	7,1
High	143	40,9
Medium	159	45,4
Low	23	6,6

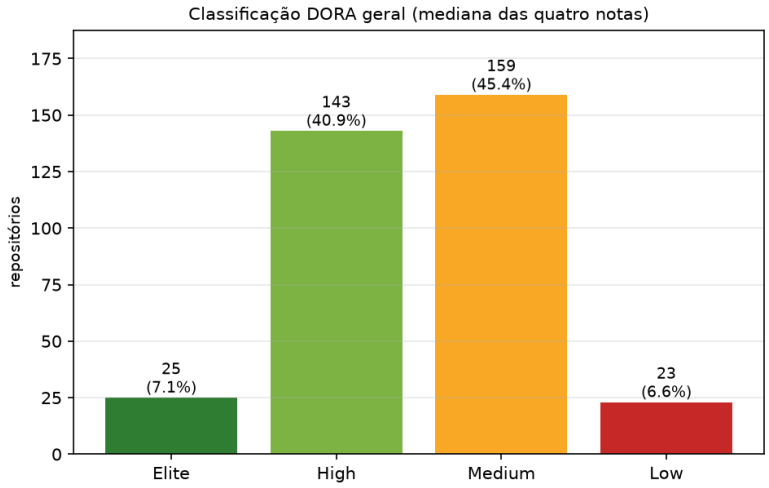


Figura 3. Classificação DORA geral dos 350 repositórios.

A amostra se concentra no meio da escala: 302 repositórios (86,3%) são *High* ou *Medium* e as duas pontas somam menos de 14%. A distribuição por nota mostra que o resultado é puxado para cima pela nota de CFR, *Elite* em 80,9% dos casos, e para baixo pela nota de frequência, *Low* ou *Medium* em 74,3%. O perfil típico da amostra é o de projetos que publicam com pouca frequência, mas com baixa taxa de falha de CI e recuperação rápida. Como a nota de CFR usa a variante (a), e como essa variante diverge fortemente da (b), essa distribuição depende da definição operacional adotada; a Discussão mede esse efeito.

3.7. RQ 05: frequência de deploy e taxa de falha

Calculamos a correlação de Spearman entre a frequência de *deploy* e o CFR, separadamente para cada variante. Spearman trabalha com postos, o que é adequado às distribuições assimétricas observadas e ao grande número de empates em zero no CFR (a).

Tabela 6. Correlação entre frequência de *deploy* e CFR, por variante.

Variante do CFR	n	ρ	p	Interpretação
(a) proxy de CI	350	0,127	0,017	fraca positiva
(b) proxy de entrega	350	0,613	$<$ 0,001	forte positiva

As duas são estatisticamente significativas ao nível de 5%, mas o coeficiente da variante (b) é quase cinco vezes o da (a). Na variante (a), um ρ de 0,127 explica menos de 2% da variância conjunta: na prática, frequência de publicação e falha de CI são independentes nesta amostra. Na variante (b), o ρ de 0,613 indica associação forte. A Figura 4 mostra as duas dispersões, com a frequência em escala logarítmica.

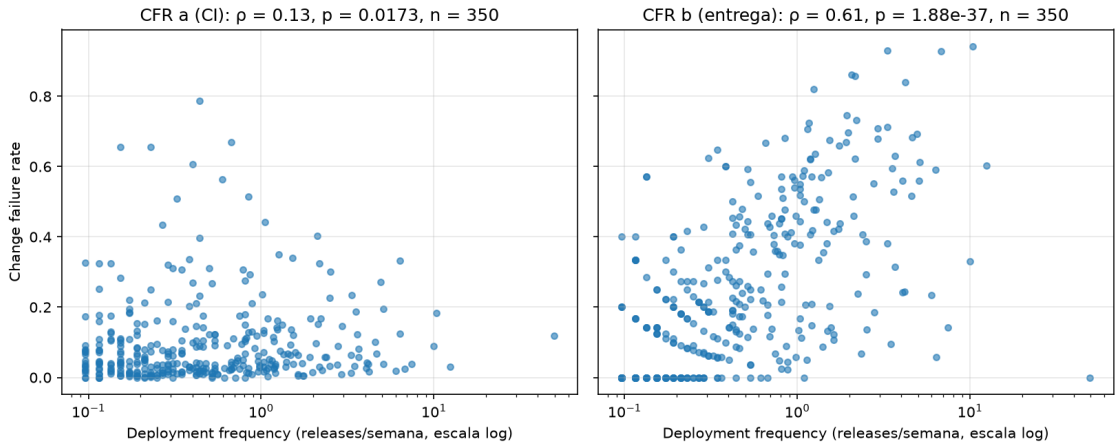


Figura 4. Frequência de *deploy* (escala log) contra o CFR, nas duas variantes.

3.8. RQ 06: fatores associados ao desempenho

Comparamos cada métrica entre subgrupos de cinco fatores: linguagem (as cinco mais frequentes, e "Outras"), quartis de estrelas, quartis de contribuidores, quartis de idade e tipo do projeto. Usamos Kruskal-Wallis para três ou mais grupos e Mann-Whitney para dois, com ε^2 e Cliff's delta como tamanho de efeito. Como são 30 testes simultâneos, aplicamos a correção de Holm [Holm 1979] sobre o conjunto.

Doze dos 30 testes permanecem significativos após a correção (Tabela 7).

Por fator, a idade aparece em cinco métricas, a linguagem em quatro, os contribuidores em duas e as estrelas em uma. Nenhum teste sobre o tipo do projeto sobreviveu à correção, e esse fator só existe para os 60 repositórios da amostra-ouro, de modo que os testes sobre ele têm poder reduzido.

Entre os testes significativos, ε^2 vai de 0,049 a 0,288; o maior corresponde à idade sobre o *lead time* (a). Nenhum fator isolado explica a maior parte da variação observada.

Tabela 7. Testes significativos após a correção de Holm, ordenados pelo tamanho de efeito.

Fator	Métrica	p (Holm)	ε^2
Idade	Lead time (a)	$<$ 0,001	0,288
Idade	Lead time (b)	$<$ 0,001	0,251
Contribuidores	Lead time (b)	$<$ 0,001	0,171
Idade	CFR (b)	$<$ 0,001	0,164
Idade	Frequência	$<$ 0,001	0,106
Contribuidores	Lead time (a)	$<$ 0,001	0,104
Linguagem	Lead time (a)	$<$ 0,001	0,099
Linguagem	Frequência	$<$ 0,001	0,097
Linguagem	CFR (b)	$<$ 0,001	0,084
Linguagem	Lead time (b)	0,001	0,078
Idade	Recuperação	0,016	0,051
Estrelas	Lead time (b)	0,014	0,049

3.9. RQ 07: sensibilidade à definição de deploy

Recalculamos a classificação DORA dos 350 repositórios sob três operacionalizações de *deploy*: C1 (referência), que usa apenas *releases*, com *lead time* (a) e CFR (a); C2, que inclui pré-*releases* e adota as variantes (b); e C3, que usa *tags* com o CFR na variante (a).

Tabela 8. Distribuição das categorias DORA sob as três definições de *deploy*.

Categoria	C1	C2	C3
Elite	25	8	33
High	143	134	164
Medium	159	186	138
Low	23	22	15

A concordância entre as classificações, medida pelo kappa ponderado, fica na faixa moderada (Tabela 9). Entre um quarto e um terço dos repositórios trocam de categoria conforme a definição adotada. A faixa *Elite* é a mais sensível: vai de 8 repositórios em C2 a 33 em C3, uma variação de quatro vezes sobre o mesmo conjunto de projetos e a mesma janela de observação.

Tabela 9. Concordância entre as classificações, par a par.

Par	Repositórios que mudam	Kappa ponderado
C1 $\times$ C2	32,9%	0,515
C1 $\times$ C3	25,4%	0,657
C2 $\times$ C3	28,0%	0,602

3.10. RQ 08: rework rate

O *rework rate* é a proporção de *releases* corretivas publicadas na janela, calculada como `cfrb_releases_corretivas` dividido por `n_releases`. Diferente do CFR (b), ele conta toda *release* corretiva, e não apenas as que vieram em até 7 dias depois de outra.

Tabela 10. *Rework rate* e CFR (b) nos 350 repositórios.

Métrica	n	Q1	Mediana	Q3
Rework rate	350	0,225	0,500	0,726
CFR (b)	350	0,052	0,200	0,416

A mediana do *rework rate* é 2,5 vezes a do CFR (b). As duas medidas se correlacionam ($\rho = 0{,}522$, $p < 0{,}001$), mas 309 dos 350 repositórios têm *rework rate* maior que o CFR (b), e 21 não publicaram nenhuma *release* corretiva na janela.

Metade das *releases* desses projetos corrige alguma coisa, mas apenas uma em cada cinco corrige algo publicado na semana anterior. A maior parte do retrabalho é manutenção de rotina, e não resposta a uma entrega recém-quebrada. Ao exigir a janela de 7 dias, o CFR (b) mede um fenômeno mais estreito que "*release* corretiva".

4. Discussão

4.1. As hipóteses sobre a taxa de falha erraram nas duas direções

A Introdução previa que o CFR na variante (a) ficaria entre 10% e 25%. O valor observado foi de 4,9%, menos da metade do piso previsto. A explicação está na definição operacional: os *runs* contabilizados são disparados por *push* no *default branch*, ou seja, mudanças que já passaram pela revisão de um *pull request* antes do *merge*. A métrica captura a taxa de falha do que já foi aprovado, e não a de todas as mudanças propostas. Em projetos open-source populares, com revisão obrigatória e CI rodando sobre o PR, essa taxa é naturalmente baixa.

Para a variante (b) a previsão era de valores parecidos ou menores e concentração em 0%. O observado foi o oposto: mediana de 0,200, quatro vezes maior que a da variante (a), ainda que 80 repositórios (22,9%) de fato fiquem em zero. Parte da diferença vem do instrumento: a heurística que decide o que é uma *release* corretiva não atinge o F1 mínimo de 0,70 exigido, alcançando 0,65 na versão adotada. A direção do erro, porém, é conhecida: a heurística ainda produz mais falsos positivos (57) do que falsos negativos (30), de modo que 0,200 tende a superestimar o valor real.

A única hipótese confirmada sobre a RQ 03 foi a de que as duas variantes concordariam pouco, e ela se confirmou de forma mais forte que o previsto.

4.2. As duas variantes do CFR medem coisas diferentes

O trabalho mediu duas vezes a mesma métrica DORA, *change failure rate*, com dois *proxies* distintos. Além da diferença de magnitude, a correlação de Spearman entre as variantes é de 0,107 ($p = 0{,}045$), praticamente nula: elas não ordenam os repositórios da mesma forma, e um projeto bem avaliado por uma pode ser mal avaliado pela outra.

O efeito sobre a conclusão é direto. A classificação DORA geral usa a nota do CFR (a). Recalculando-a com a variante (b), 72 dos 350 repositórios (20,6%) mudam de categoria, e a faixa *Elite* cai de 25 para 8. A distribuição principal (7,1% *Elite* e 40,9% *High*) depende, portanto, da definição operacional escolhida, e não só dos repositórios estudados.

Na RQ 02 foi diferente: as duas variantes do *lead time* produziram medianas próximas (251,7 h e 284,0 h) e correlação de 0,744. A escolha entre operacionalizações importa pouco para o *lead time* e muito para o CFR. As variantes do *lead time* medem o mesmo evento por janelas diferentes, enquanto as do CFR medem eventos distintos: falha de *pipeline* antes da entrega e necessidade de correção depois dela.

4.3. A escala DORA não foi desenhada para releases de projetos open-source

A frequência de deploy é a dimensão em que a amostra vai pior: 74,3% dos repositórios publicam menos de uma *release* por semana e apenas 5 dos 350 alcançam o patamar *Elite*.

Esse resultado reflete mais a distância entre o conceito e o *proxy* do que a capacidade de entrega desses projetos. Uma *release* publicada no GitHub é um marco de versionamento, decidido pelos mantenedores e frequentemente mensal ou trimestral. Um *deploy*, no sentido do DORA [DORA 2024], é a colocação de uma mudança em produção, que em muitos desses projetos acontece continuamente e sem gerar *release*. A faixa *Elite*, de sete *deploys* por semana, pressupõe entrega contínua em ambiente corporativo, o contexto em que as faixas foram estabelecidas [Forsgren et al. 2018]; aplicá-la a marcos de versionamento classifica como lentos projetos que apenas versionam com parcimônia.

4.4. A classificação geral é dominada por uma única nota

A categoria final é a mediana das quatro notas, e elas não contribuem igualmente: a nota de CFR é *Elite* em 80,9% dos casos, porque o corte *Elite* (CFR $\leq$ 0,15) é generoso diante de uma mediana observada de 0,049. A nota de frequência, no sentido oposto, é *Low* ou *Medium* em 74,3%.

O perfil típico da amostra é o de projetos que publicam com pouca frequência, mas com baixa taxa de falha de CI e recuperação rápida (mediana de 2,56 h, com 59,4% na faixa *High*). Como a mediana de quatro notas é pouco sensível a uma delas, a classificação geral acaba determinada pelo par CFR/recuperação, as dimensões em que a amostra vai melhor.

4.5. Velocidade e estabilidade (RQ 05)

A hipótese previa correlação fraca negativa ou nula entre frequência de *deploy* e CFR na variante (a), e positiva na variante (b). As duas previsões se confirmaram na direção, e a segunda com força muito maior que a esperada.

Na variante (a), a correlação de Spearman é de $\rho = 0{,}127$ ($p = 0{,}017$, $n = 350$): positiva, mas fraca a ponto de não ter relevância prática. Não há evidência de que publicar com mais frequência venha acompanhado de mais falhas de CI, o que é compatível com a tese do DORA de que velocidade e estabilidade não constituem um *trade-off*.

Na variante (b), a correlação é de $\rho = 0{,}613$ ($p < 0{,}001$): forte e positiva. Tomada ao pé da letra, ela diria que os projetos que publicam mais falham muito mais. A hipótese já antecipava que esse resultado seria, em boa medida, um artefato da própria métrica, e a explicação se sustenta: o CFR (b) marca uma *release* como falha quando a *release* seguinte, publicada em até 7 dias, é corretiva. Quem publica semanalmente tem muito mais oportunidades de satisfazer essa condição do que quem publica trimestralmente, independentemente da qualidade do que entrega.

Por isso reportamos as duas variantes em separado. Com apenas a (b), a conclusão seria o oposto do que o DORA sustenta, e por causa da construção da métrica, não do comportamento dos projetos.

4.6. O que explica as diferenças entre projetos (RQ 06)

Dos 30 testes de Kruskal-Wallis, 12 permanecem significativos após a correção de Holm. Os tamanhos de efeito desses 12, medidos por ε^2, vão de 0,049 a 0,288.

A idade do repositório é o fator mais forte, e aparece em cinco das seis métricas. O efeito mais pronunciado é sobre o *lead time* ($\varepsilon^2 = 0{,}288$ na variante (a) e 0,251 na (b)): projetos mais antigos acumulam mais tempo entre o *commit* mais antigo e a publicação. A idade também se associa ao CFR (b) ($\varepsilon^2 = 0{,}164$), a frequência de *deploy* ($\varepsilon^2 = 0{,}106$) e, com efeito menor, o tempo de recuperação ($\varepsilon^2 = 0{,}051$).

O número de contribuidores se associa às duas variantes do *lead time* ($\varepsilon^2 = 0{,}171$ na (b) e 0,104 na (a)), e a linguagem aparece em quatro métricas, com ε^2 entre 0,078 e 0,099. A popularidade, medida por quartis de estrelas, só sobrevive à correção em uma métrica, o *lead time* (b), com o menor efeito de todos ($\varepsilon^2 = 0{,}049$): o número de estrelas quase não explica o desempenho de entrega.

Nenhum teste sobre o tipo do projeto foi significativo. Esse fator, porém, só existe para os 60 repositórios da amostra-ouro, e os testes sobre ele têm poder reduzido: a ausência de significância ali não deve ser lida como ausência de efeito.

4.7. A definição de deploy muda a conclusão? (RQ 07 e RQ 08)

Entre um quarto e um terço dos repositórios trocam de categoria conforme a definição de *deploy* adotada (Tabela 9), e a concordância entre as classificações é apenas moderada. A faixa *Elite* é a mais instável: 25 repositórios em C1, 8 em C2 e 33 em C3, uma variação de quatro vezes entre os extremos sobre o mesmo conjunto de projetos e a mesma janela.

A RQ 08 ajuda a ler o CFR (b). O *rework rate* conta toda *release* corretiva e tem mediana de 0,500; o CFR (b), que só conta correções em até 7 dias, fica em 0,200. Ou seja, a maior parte das correções nesses projetos é manutenção de rotina, e o CFR (b) captura só a fração que responde a uma entrega recente.

5. Ameaças à Validade

Seguimos as quatro categorias de Wohlin et al. [Wohlin et al. 2012].

5.1. Validade de construto

As ameaças mais sérias do trabalho estão nesta categoria.

Release não é deploy. Muitos projetos entregam continuamente sem publicar *release*, e outros publicam *releases* que são apenas marcos de versionamento. A validação manual mediu esse risco diretamente: os três avaliadores julgaram, para cada repositório da amostra-ouro, se as *releases* representam entregas reais ao usuário, e o consenso foi afirmativo em 59 dos 60 casos. É um indício favorável, mas limitado a 60 repositórios populares.

Falha de CI não é falha em produção. Um *workflow run* que falha é, na maior parte das vezes, um erro capturado antes de chegar ao usuário, que é a finalidade do CI.

Contá-lo como *change failure* inverte o sentido da métrica DORA, que mede falhas que atingiram produção. É a explicação mais provável para o CFR (a) observado ter ficado abaixo do previsto.

As duas variantes do CFR não medem a mesma coisa. A correlação entre elas é de 0,107 e 20,6% dos repositórios mudam de categoria conforme a variante. Qualquer afirmação sobre a taxa de falha da amostra é condicional à variante escolhida.

A heurística de release corretiva não atinge o limiar exigido. As versões testadas ficaram entre 0,56 e 0,65 de F1, abaixo do mínimo de 0,70; a versão adotada, a v4, alcançou 0,65. O CFR (b), a RQ 08 e o C2 da RQ 07 herdam esse erro de medida. Mesmo os avaliadores humanos não foram unânimes nessa dimensão: o kappa de Fleiss para *release* corretiva foi de 0,876, com 26 divergências em 299 itens, o que funciona como teto prático para qualquer heurística avaliada contra esse consenso.

O `commit.author.date` é distorcido por rebase e squash merge. Essas operações reescrevem a data do commit e afetam as duas variantes do *lead time*. Não há como detectá-las pela API, e o efeito é de direção desconhecida.

5.2. Validade interna

As análises são observacionais e correlacionais; nenhuma afirmação causal é feita.

Medimos as associações entre metadados e métricas e encontramos relações não desprezíveis: a idade do repositório correlaciona-se negativamente com a frequência de deploy ($\rho = -0{,}306$, $p < 0{,}001$), e o número de estrelas ($\rho = 0{,}162$) e de contribuidores ($\rho = 0{,}119$) positivamente. Projetos mais antigos publicam menos *releases* na janela; os mais populares publicam mais. Esses fatores não são controlados nas RQ 01 a RQ 04, que são descritivas; a RQ 06 os trata explicitamente.

O filtro de inclusão introduz seleção: só entraram repositórios com ao menos 5 *releases* e 50 *runs* válidos na janela. Dos 1.057 candidatos que usam Actions, 618 foram descartados por publicar menos de 5 *releases* e 81 por ter menos de 50 *runs*. A amostra final é, por construção, composta de projetos que já versionam e já automatizam com regularidade, o que restringe a variação na dimensão da RQ 01.

Seis repositórios foram descartados por ultrapassarem 1.000 *runs* em um único dia, limite acima do qual a API não devolve o conjunto completo. Preferimos excluí-los a calcular métricas sobre um subconjunto truncado de tamanho desconhecido, mas isso remove da amostra os projetos de maior atividade de CI. Outros dois saíram por erro definitivo da API.

5.3. Validade externa

A limitação mais importante é que a amostra não é aleatória. A busca do GitHub devolve no máximo 1.000 resultados por consulta, e cada faixa de estrelas contém muito mais repositórios que isso (a faixa de 1.000 a 1.500 estrelas sozinha tem mais de 20 mil). Entraram os 1.000 mais estrelados de cada uma das sete faixas, e não um sorteio delas. Fatiar a busca em faixas estreitas atenua o viés, mas não o elimina: dentro de qualquer faixa com mais de 1.000 repositórios, os menos estrelados nunca são observados.

A amostra deve ser lida como "repositórios populares que usam GitHub Actions", e não como "repositórios com mais de 1.000 estrelas". Projetos privados e corporativos

estão fora, e são o contexto para o qual as faixas DORA foram concebidas. Nenhuma conclusão aqui se transfere automaticamente para equipes corporativas de entrega contínua.

A janela de observação é única e fixa, de 12 meses. Não há como separar o que é característica estável de um projeto do que é efeito do período: um projeto pode ter passado por migração de CI, reescrita ou troca de mantenedores dentro da janela.

5.4. Validade de conclusão

Usamos mediana e IQR em vez de média e desvio padrão, porque todas as distribuições são assimétricas com cauda longa e a média seria dominada pelos extremos. Nas correlações usamos Spearman, que trabalha com postos e não pressupõe normalidade nem linearidade; o grande número de empates em zero no CFR (a) é tratado pela média dos postos.

Nos testes por subgrupo da RQ 06 são feitas 30 comparações simultâneas, o que inflaria o erro tipo I; aplicamos a correção de Holm [Holm 1979]. Os tamanhos de efeito são reportados junto dos p-valores (ε^2 e Cliff's delta), porque significância estatística com $n = 350$ não implica relevância prática.

Alguns subgrupos são pequenos. O tipo do projeto, usado como fator na RQ 06, só existe para os 60 repositórios da amostra-ouro, e os testes sobre ele têm poder reduzido: um resultado não significativo ali não deve ser lido como ausência de efeito.

O tempo de recuperação é calculado sobre 330 repositórios; os 20 sem episódio na janela são reportados à parte, e não imputados. A censura dentro da janela mostrou-se desprezível (70,9% dos repositórios não têm nenhum episódio censurado), o que reduz o risco de subestimação da mediana.

Por fim, o kappa de "releases são entregas reais" (0,745) deve ser lido junto com a distribuição dos rótulos: como o consenso pôs 59 dos 60 repositórios na mesma categoria, uma única divergência basta para derrubar o coeficiente, embora a concordância bruta seja de 98,3%.

6. Conclusão

Medimos as quatro métricas DORA em 350 repositórios *open-source* populares usando somente dados públicos do GitHub. A amostra publica com pouca frequência (mediana de 0,42 *release* por semana, com 74,3% abaixo de uma por semana), leva cerca de dez dias entre o *commit* mais antigo e a publicação, registra baixa taxa de falha de integração contínua (mediana de 0,049) e se recupera rápido (mediana de 2,56 h, com 59,4% na faixa *High*). A classificação geral concentra-se no meio da escala: 86,3% dos repositórios são *High* ou *Medium*.

Esses valores dependem bastante de decisões de medição. As duas operacionalizações da taxa de falha correlacionam-se a $\rho = 0{,}107$ e discordam sobre a categoria de 20,6% da amostra; trocar a definição de *deploy* move entre 25% e 33% dos repositórios, e a faixa *Elite* varia de 8 a 33 projetos conforme a combinação adotada. A correlação entre frequência e taxa de falha é praticamente nula na variante de CI ($\rho = 0{,}127$) e forte na variante de entrega ($\rho = 0{,}613$), em boa parte por artefato da construção da métrica.

Uma classificação DORA de projetos *open-source* só pode ser interpretada junto com as escolhas operacionais que a produziram.

Na validação manual, o *proxy* de *release* se sustentou: o consenso dos três avaliadores considerou que as *releases* representam entregas reais em 59 dos 60 repositórios da amostra-ouro. Já a melhor heurística automática para identificar *releases* corretivas alcançou F1 de 0,65, abaixo do limiar de 0,70, em uma dimensão na qual os próprios avaliadores divergiram em 26 de 299 casos.

Como trabalhos futuros, sugerimos três caminhos: usar projetos que publicam ambiente de produção observável (por exemplo, imagens de contêiner com data de implantação) para substituir o *proxy* de *release*; estender a janela para além de 12 meses, separando característica estável de efeito de período; e ampliar a amostra-ouro, de modo a permitir treinar e validar a heurística de *release* corretiva em conjuntos disjuntos.

Disponibilidade dos dados

Todo o código, os dados coletados e os roteiros de análise estão em `https://github.com/candian15/LabExpGRP`, na pasta `LAB03`. A coleta é reproduzida por um único comando, e o dicionário de dados descreve cada coluna dos CSVs.

Referências

DORA (2024). DORA's software delivery metrics: the four keys. `https://dora.dev/guides/dora-metrics-four-keys/`. Acesso em 2026.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, Portland, OR.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg.